

Interview Questions — Java - Spring

Total Questions: 40

1. What is the Spring Framework, and what problem does it aim to solve for Java developers?

Hint: Think about how Spring changes the way Java applications are structured and managed.

2. Explain the main differences between traditional Servlets/JSP development and using the Spring Framework.

Hint: Focus on how configuration, object creation, and modularity are handled.

3. Name any two core modules of the Spring Framework and briefly describe their purpose.
-

4. What is Inversion of Control (IoC) in the context of Spring?

Hint: Consider who is responsible for creating and managing objects.

5. Why does Dependency Injection (DI) improve modularity in Java applications?

Hint: Think about how dependencies are provided to classes and how this affects testing or swapping implementations.

6. Describe the difference between setter injection and constructor injection in Spring.

Hint: Focus on how dependencies are passed to the bean.

7. In a Spring application, how would you inject a dependency into a StudentService bean using XML configuration?
-

8. How do you define a bean in applicationContext.xml in Spring?

Hint: Mention the main XML tag and required attributes.

9. What is the difference between singleton and prototype bean scopes in Spring?

Hint: Consider how many instances Spring creates for each scope.

10. Explain what autowiring is in Spring and how it helps with dependency management.
-

11. If you have two beans of the same type in your Spring configuration, how does Spring decide which one to autowire?

Hint: Think about what happens if there is ambiguity.

12. How would you configure multiple beans in Spring and access them using the IoC container?

13. What is the purpose of the @Component annotation in Spring?

Hint: Consider what happens to a class when it is annotated this way.

14. How does @Autowired help in annotation-based configuration?

Hint: Think about how dependencies are injected automatically.

15. What is the difference between using @Bean in a @Configuration class and using @Component on a class?

Hint: Focus on how beans are registered in the Spring context.

16. How would you convert an XML-based configuration to annotation-based configuration in a Spring project?

17. Show how you would annotate a service class with @Component and inject a dependency using @Autowired.

18. What is JdbcTemplate in Spring, and why is it commonly used for database operations?

Hint: Consider how it simplifies JDBC code.

19. Describe how you would configure a datasource in a Spring application for JDBC access.

20. Write a simple method using JdbcTemplate to insert a new Student record into the database.

21. How would you organize the DAO layer in a Spring application using Spring JDBC?

22. What steps are involved in performing a CRUD operation (like update or delete) on a Student table using Spring JDBC?

23. Explain the role of DispatcherServlet in Spring MVC.

Hint: Think about what happens when a web request comes in.

24. What is the purpose of a Controller in Spring MVC?

Hint: Consider how requests are processed and responses generated.

25. How does Spring MVC handle GET and POST requests differently in a controller method?

26. What is ModelAndView in Spring MVC, and how is it used in controllers?

Hint: Think about sharing data between controller and view.

27. Describe the flow of a simple 'Add Event' form submission in a Spring MVC application.

28. Show how you would map a URL to a controller method using @RequestMapping in Spring MVC.

29. When integrating a feature like 'Event Management' into a Spring MVC module, which annotations would you typically use in your controller class?

30. How would you render a JSP view from a Spring MVC controller after processing form data?

31. Imagine you are building the 'Event Management' module for Campus Connect using Spring MVC. Briefly outline the main components (controller, view, service) and how they interact.

32. Suppose your Spring MVC form is not displaying validation errors as expected. What steps would you take to troubleshoot the issue?

Hint: Think about controller logic, model attributes, and the JSP view.

33. How would you use annotation-based configuration to wire up both your DAO and service layers in a Spring MVC mini-project?

34. If you have to switch from XML-based bean configuration to annotation-based configuration in an existing Spring MVC module, what are the key changes you would make?

35. Explain how you would handle form validation in a Spring MVC controller.

36. How could you use an AI tool like ChatGPT or GitHub Copilot to help you write or debug a Spring bean configuration, and how would you ensure the generated code is correct?

Hint: Think about verifying code output and checking against official documentation.

-
- 37.** In what ways could AI tools help speed up development when building a Spring MVC project, and what are some things you should always double-check before using AI-generated code in production?
-
- 38.** Integrative: How does Dependency Injection (Session 2) make it easier to test components in a Spring MVC application (Session 6/7)?
- Hint:** Consider how dependencies can be swapped or mocked.
-
- 39.** Integrative: When performing CRUD operations on a Student table (Session 5), how would you use annotation-based configuration (Session 4) to wire up your DAO and service beans?
-
- 40.** Integrative: Explain how the DispatcherServlet (Session 6) works together with beans defined in your application context (Session 3) to process a web request.
-